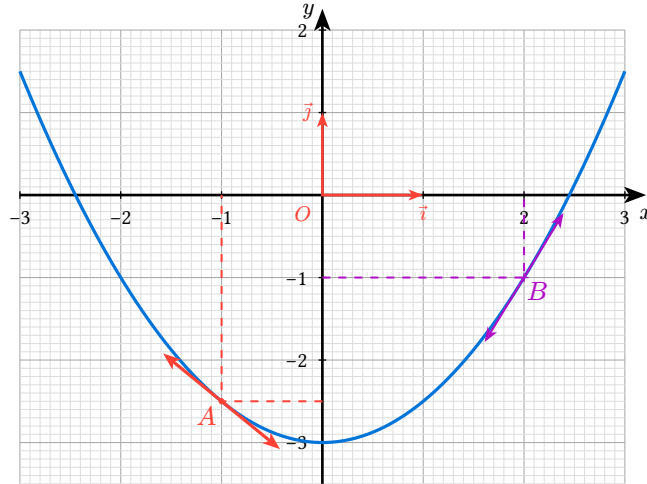


ctz-plot Helper

– Quick reference –



- `anchored-basis-vectors()`
- `anchored-derivative-arrow()`
- `anchored-lines()`
- `anchored-point()`
- `anchored-point-marker()`
- `draw-mark-shape()`
- `monplot()`
- `newFig()`
- `plot-scale()`
- `point-marker()`
- `resolve-origine()`
- `rotate-point()`
- `setAxes()`
- `setExtraStyles()`
- `setPlot()`

Variables

- `mesaxes`

anchored-basis-vectors

Trace les deux vecteurs de base (O ; $\text{vec}(i)$, $\text{vec}(j)$), avec une longueur de `length` unités de `DONNÉES` (défaut 1 = vecteur unitaire). Hampe et pointe sont dessinées directement dans le repère du canvas : pas de déformation liée à l'anisotropie du repère. Les deux segments forment UNE seule polyligne continue (i) – (O) – (j) : aucun « décroché » à l'origine. À appeler EN DEHORS de `plot.plot(...)`. Récupère automatiquement `sx`, `sy` (calculés par `newFig`) : rien à retourner.

- `origine` (str, array): ancre "plot." ("plot.O" par défaut), ou (`x`, `y`) en unités de données
- `x-length` (float, int): longueur de $\text{vec}(i)$, en unités de données
- `y-length` (float, int): longueur de $\text{vec}(j)$, en unités de données
- `fill` (color): couleur des flèches et des labels

- stroke (auto, stroke): trait (auto = fill + 1pt)
- mark-scale (float, int): taille des pointes de flèche

Parameters

```
anchored-basis-vectors(
  origine: string array,
  x-length: float int,
  y-length: float int,
  fill: color,
  stroke: auto stroke,
  mark-scale: float int
) -> content
```

origine string or array

Ancre “plot.” (“plot.O” par défaut), ou (x, y) en unités de données.

Default: "plot.O"

x-length float or int

Longueur de vec(i), en unités de données.

Default: 1

y-length float or int

Longueur de vec(j), en unités de données.

Default: 1

fill color

Couleur des flèches et des labels.

Default: red

stroke auto or stroke

Trait (auto = fill + 1pt).

Default: auto

mark-scale float or int

Taille des pointes de flèche.

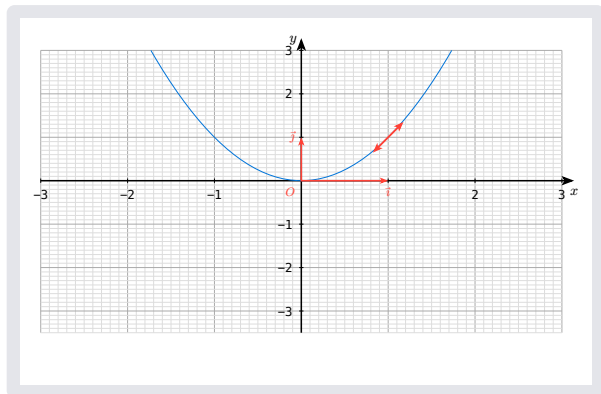
Default: .5

anchored-derivative-arrow

Flèche de dérivée à taille FIXE (cm), avec la pente corrigée pour rester géométriquement cohérente dans un repère anisotrope. À appeler EN DEHORS de `plot.plot(...)`. Récupère automatiquement `sx, sy` (calculés par `newFig`) : rien à retourner.

- `origine` (str, array): ancre "plot.", ou (x, y) en unités de données
- `slope` (float, int): pente RÉELLE en unités de données (ex. `spline-dy(...)`)
- `fill` (color): couleur de la flèche
- `length` (float, int): longueur FIXE de la flèche, en cm
- `stroke` (stroke): épaisseur/couleur du trait
- `mark-scale` (float, int): taille de la pointe de flèche

```
#newFig(  
  extra: (sx, sy) => {  
    import draw: *  
    // tangente au point (1, f(1)) de  
    pente 2  
    anchored-derivative-arrow((1, 1), 2,  
  fill: red, stroke: red + 1.2pt)  
  },  
  { plot.add(domain: (-2, 2), x =>  
    calc.pow(x, 2), style: (stroke: blue +  
    0.5pt), samples: 50) },  
)
```



Parameters

```
anchored-derivative-arrow(  
  origine: string array,  
  slope: float int,  
  fill: color,  
  length: float int,  
  stroke: stroke,  
  mark-scale: float int  
) -> content
```

origine string or array

Ancre "plot.", ou (x, y) en unités de données.

slope float or int

Pente RÉELLE en unités de données (ex. `spline-dy(...)`).

fill color

Couleur de la flèche.

Default: black

length float or int

Longueur FIXE de la flèche, en cm.

Default: 1

stroke stroke

Épaisseur/couleur du trait.

Default: 1pt

mark-scale float or int

Taille de la pointe de flèche.

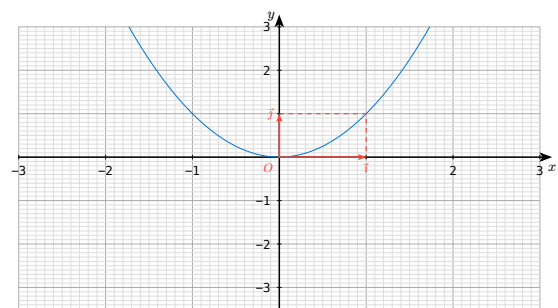
Default: .6

anchored-lines

Trace une ligne brisée (segments successifs) entre plusieurs points du repère (O ; i, j) — chaque point est une ancre “plot.” ou des coordonnées (x, y) en unités de données, comme pour resolve-origine. Les points sont convertis en coordonnées canvas AVANT d’être reliés, donc le trait reste rectiligne à l’écran même dans un repère anisotrope (contrairement à une ligne tracée en unités de données à l’intérieur du plot). À appeler EN DEHORS de plot.plot(...). Récupère automatiquement sx, sy (calculés par newFig) : rien à retourner.

- points (str | array): ancres “plot.” et/ou coordonnées (x, y), au moins deux
- stroke (stroke): trait de la ligne
- mark (none | dictionary): pointes de flèche aux extrémités (voir line(mark:))

```
#newFig(  
  extra: (sx, sy) => {  
    import draw: *  
    // pointillés (0 ; f) -- (x ; f) --  
    (x ; 0)  
    anchored-lines((0, 1), (1, 1), (1, 0),  
stroke: (thickness: 0.75pt, dash:  
"dashed", paint: red))  
  },  
  { plot.add(domain: (-2, 2), x =>  
calc.pow(x, 2), style: (stroke: blue +  
0.5pt), samples: 50) },  
)
```



Parameters

```
anchored-lines(  
  ..points: string array,  
  stroke: stroke,  
  mark: none dictionary  
) -> content
```

..points `string` or `array`

Ancres “plot.” et/ou coordonnées (x, y), au moins deux.

stroke `stroke`

Trait de la ligne.

Default: black + `1pt`

mark `none` or `dictionary`

Pointes de flèche aux extrémités (voir `line(mark:)`).

Default: `none`

anchored-point

Cas particulier de `anchored-point-marker` : un simple rond. Gardé pour compatibilité — préfère `anchored-point-marker` si tu veux choisir la forme.

- `origine` (str, array): ancre “plot.”, ou (x, y) en unités de données
- `radius` (float, int): rayon du rond, en cm
- `fill` (color): couleur de remplissage
- `stroke` (none, stroke, color): contour (none = pas de contour)

Parameters

```
anchored-point(  
  origine: string array ,  
  radius: float int ,  
  fill: color ,  
  stroke: none stroke color  
) -> content
```

origine `string` or `array`

Ancre “plot.”, ou (x, y) en unités de données.

radius `float` or `int`

Rayon du rond, en cm.

Default: `0.1`

fill `color`

Couleur de remplissage.

Default: red

stroke `none` or `stroke` or `color`

Contour (none = pas de contour).

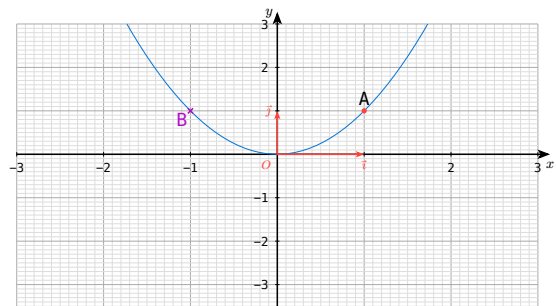
Default: `none`

anchored-point-marker

Marqueur et/ou étiquette optionnels à une ancre du plot OU à des coordonnées (x, y) en unités du repère (O ; i, j) — voir `resolve-origine`. Récupère automatiquement `sx, sy` (calculés par `newFig`). À appeler EN DEHORS de `plot.plot(...)`. Chaque objet est indépendant : si `marker` ou `label` vaut `none`, on ne le dessine pas (les deux à `none` → rien du tout).

- `origine` (str, array): ancre “plot.”, ou (x, y) en unités de données
- `marker` (dictionary, none): (marker-symbol, marker-size, marker-fill, marker-stroke, marker-angle) — voir `draw-mark-shape` ; `none` = pas de point
- `label` (dictionary, none): (label-text, label-distance, label-anchor, label-position) ; `none` = pas d’étiquette

```
#newFig(  
  extra: (sx, sy) => {  
    import draw: *  
    // marqueur + étiquette (chacun est  
    optionnel)  
    anchored-point-marker((1, 1), marker:  
      (marker-fill: red), label: (label-text:  
        "A", label-position: 90deg))  
    // étiquette seule, sans point  
    anchored-point-marker((-1, 1), marker:  
      (marker-symbol: "x", marker-size: 0.06,  
        marker-stroke: 1pt+purple), label: (label-  
        text: text(fill: purple)[B], label-  
        position: -135deg))  
  },  
  { plot.add(domain: (-2, 2), x =>  
    calc.pow(x, 2), style: (stroke: blue +  
      0.5pt), samples: 50) },  
)
```



Parameters

```
anchored-point-marker(  
  origine: string array,  
  marker: dictionary none,  
  label: dictionary none  
) -> content
```

origine `string` or `array`

Ancre “plot.”, ou (x, y) en unités de données.

marker dictionary or **none**

(marker-symbol, marker-size, marker-fill, marker-stroke, marker-angle) — voir draw-mark-shape ; none = pas de point.

Default: (marker-symbol: "o", marker-size: 0.06, marker-fill: red, marker-stroke: none, marker-angle: 0deg)

label dictionary or **none**

(label-text, label-distance, label-anchor, label-position) ; none = pas d'étiquette.

Default: (label-text: "", label-distance: 8pt, label-anchor: "center", label-position: 0deg)

draw-mark-shape

Dessine la “marque” d’un point à une position DÉJÀ EN COORDONNÉES DU CANVAS (x, y en cm).
Choix de forme partagé entre point-marker (dessine en unités de données, dans plot.annotate) et anchored-point-marker (dessine en cm, hors du plot). Ne pas appeler directement en temps normal.

- x (float, int): abscisse du point, en unités canvas
- y (float, int): ordonnée du point, en unités canvas
- symbol (str): “o” (rond), “x” (croix), “+” (plus), “square” (carré), “triangle”, “diamond” (losange)
- size (float, int): demi-taille du symbole, dans la même unité que x, y
- fill (color): couleur de remplissage (formes fermées) ou du trait (“+”/“x”)
- stroke (none, stroke, color): contour des formes fermées (none = pas de contour, juste fill)
- angle (angle): orientation (pertinent pour “square”, “triangle”, “diamond”, “+”, “x” — sans effet sur “o”)

Parameters

```
draw-mark-shape(  
    x: float int,  
    y: float int,  
    symbol: string,  
    size: float int,  
    fill: color,  
    stroke: none stroke color,  
    angle: angle  
) -> content
```

x float or int

Abscisse du point, en unités canvas.

y float or int

Ordonnée du point, en unités canvas.

symbol `string`

“o” (rond), “x” (croix), “+” (plus), “square” (carré), “triangle”, “diamond” (losange).

Default: `"o"`

size `float` or `int`

Demi-taille du symbole, dans la même unité que x, y.

Default: `0.06`

fill `color`

Couleur de remplissage (formes fermées) ou du trait (“+”/“x”).

Default: `red`

stroke `none` or `stroke` or `color`

Contour des formes fermées (none = pas de contour, juste fill).

Default: `none`

angle `angle`

Orientation (pertinent pour “square”, “triangle”, “diamond”, “+”, “x” — sans effet sur “o”).

Default: `0deg`

monplot

Encapsule `plot.plot(...)` : reçoit un dictionnaire d’options déjà prêt (typiquement produit par `setPlot(...)`) et le body qui va DANS le plot (tes `plot.add(...)`, `plot.add-anchor(...)`, `plot.annotate(...)`). Ajoute automatiquement `plot.add-anchor("0", (0, 0))` : tu n’as plus besoin de le faire toi-même, l’ancre “plot.O” est toujours disponible pour `anchored-basis-vectors(...)` etc.

- options (dictionary): options de `plot.plot`, typiquement produites par `setPlot(...)`
- body (content): contenu placé dans `plot.plot` (`plot.add`, `plot.add-anchor`, `plot.annotate...`)

Parameters

```
monplot(  
    options: dictionary,  
    body: content  
) -> content
```

options `dictionary`

Options de `plot.plot`, typiquement produites par `setPlot(...)`.

body **content**

Contenu placé dans plot.plot (plot.add, plot.add-anchor, plot.annotate...).

newFig

Fonction “tout-en-un” pour construire une figure : calcule s_x , s_y automatiquement (plus besoin d’appeler plot-scale toi-même), ouvre le canvas et applique le style des axes, appelle monplot(...) avec les options fusionnées de setPlot(...), et ajoute, si fourni, le contenu HORS du plot mais DANS le canvas (accès à s_x , s_y). Si repere: true (défaut), ajoute automatiquement le repère (O ; i, j) ET masque la graduation “1” sur les axes (puisqu’elle fait double emploi avec les flèches i, j) ; si repere: false, pas de flèches, et la graduation “1” redevient visible comme les autres. Un x-format/y-format donné dans plot: prend le pas sur celui piloté par repere: — mais ne mets pas les deux à la fois pour la même clé (x-format ou y-format), Typst refuse un argument nommé en double.

- size (array): dimensions physiques du plot (largeur, hauteur), en cm
- x-min (float | int): borne inférieure du domaine en x
- x-max (float | int): borne supérieure du domaine en x
- y-min (float | int): borne inférieure du domaine en y
- y-max (float | int): borne supérieure du domaine en y
- axes (dictionary): style des axes, voir setAxes()
- extraStyles (dictionary): styles des flèches de base (i, j), voir setExtraStyles()
- repere (bool): affiche/masque (O ; vec(i), vec(j)) et la graduation “1” (voir description)
- plot (dictionary): surcharge des options de setPlot() (ex. (x-tick-step: 0.5))
- extra (function, none): (s_x , s_y) => contenu, dessiné après le plot, dans le canvas
- body (content): contenu placé DANS plot.plot (plot.add, plot.add-anchor, plot.annotate...)

```
#newFig(  
  // contenu DANS plot.plot  
  {  
    plot.add(domain: (-3, 3), x => x * 0, style: (stroke: gray + 0.5pt), samples: 2)  
    plot.add-anchor("p0", (-2, -1))  
  },  
  // contenu HORS plot, dans le canvas (accès à  $s_x$ ,  $s_y$ )  
  extra: ( $s_x$ ,  $s_y$ ) => {  
    import draw: *  
    anchored-point-marker("plot.p0", marker: (marker-fill: red), label: (label-text:  
"A", label-position: 90deg))  
    anchored-basis-vectors(fill: red)  
  },  
)
```

Parameters

```
newFig(  
    size: array ,  
    x-min: float int ,  
    x-max: float int ,  
    y-min: float int ,  
    y-max: float int ,  
    axes: dictionary ,  
    extraStyles: dictionary ,  
    repere: bool ,  
    plot: dictionary ,  
    extra: function none ,  
    body: content  
) -> content
```

size array

Dimensions physiques du plot (largeur, hauteur), en cm.

Default: (12.0, 6.5)

x-min float or int

Borne inférieure du domaine en x.

Default: -3.0

x-max float or int

Borne supérieure du domaine en x.

Default: 3.0

y-min float or int

Borne inférieure du domaine en y.

Default: -3.5

y-max float or int

Borne supérieure du domaine en y.

Default: 3.0

axes dictionary

Style des axes, voir setAxes().

Default: [setAxes\(\)](#)

extraStyles dictionary

Styles des flèches de base (i, j), voir setExtraStyles().

Default: `setExtraStyles()`

repere bool

Affiche/masque (O ; vec(i), vec(j)) et la graduation “1” (voir description).

Default: `true`

plot dictionary

Surcharge des options de setPlot() (ex. (x-tick-step: 0.5)).

Default: `(:)`

extra function or none

(sx, sy) => content, dessiné après le plot, dans le canvas.

Default: `none`

body content

Contenu placé DANS plot.plot (plot.add, plot.add-anchor, plot.annotate...).

plot-scale

Facteurs d'échelle (unités canvas par unité de donnée) d'un plot dont on connaît la taille physique et le domaine (fixe) des deux axes. Sert à convertir une pente réelle en pente visuelle à l'écran.

- size (array): taille physique du plot (largeur, hauteur), en cm
- x-domain (array): domaine (x-min, x-max) de l'axe des abscisses
- y-domain (array): domaine (y-min, y-max) de l'axe des ordonnées

Parameters

```
plot-scale(  
  size: array,  
  x-domain: array,  
  y-domain: array  
) -> dictionary
```

size array

Taille physique du plot (largeur, hauteur), en cm.

x-domain array

Domaine (x-min, x-max) de l'axe des abscisses.

y-domain array

Domaine (y-min, y-max) de l'axe des ordonnées.

point-marker

Marque un point isolé (x, y) EN UNITÉS DE DONNÉES — à utiliser DANS plot.annotate (cetz-plot gère alors la mise à l'échelle lui-même). Même vocabulaire de formes que plot.add (mark:), mais sans son bug du point unique. Voir draw-mark-shape pour le détail des paramètres.

- x (float, int): abscisse du point, en unités de données
- y (float, int): ordonnée du point, en unités de données
- symbol (str): "o", "x", "+", "square", "triangle", "diamond"
- size (float, int): demi-taille du symbole, en unités de données
- fill (color): couleur de remplissage ou du trait
- stroke (none, stroke, color): contour (none = pas de contour, juste fill)
- angle (angle): orientation du symbole

Parameters

```
point-marker(  
    x: float int,  
    y: float int,  
    symbol: string,  
    size: float int,  
    fill: color,  
    stroke: none stroke color,  
    angle: angle  
) -> content
```

x float or int

Abscisse du point, en unités de données.

y float or int

Ordonnée du point, en unités de données.

symbol string

"o", "x", "+", "square", "triangle", "diamond".

Default: "o"

size float or int

Demi-taille du symbole, en unités de données.

Default: 0.06

fill color

Couleur de remplissage ou du trait.

Default: red

stroke none or stroke or color

Contour (none = pas de contour, juste fill).

Default: none

angle angle

Orientation du symbole.

Default: 0deg

resolve-origine

Résout origine en position CANVAS (cm) : soit une ancre existante (“plot.”), soit des coordonnées (x, y) en unités du repère (O ; i, j) — auquel cas on part de l’ancree “plot.O” et on applique sx, sy. Utilisé par anchored-basis-vectors, anchored-point-marker et anchored-derivative-arrow ; pas destiné à être appelé directement.

- ctx (dictionary): contexte cetz courant (fourni par get-ctx)
- origine (str, array): ancre “plot.”, ou (x, y) en unités de données
- sx (float, int): facteur d’échelle en x (cm par unité de donnée)
- sy (float, int): facteur d’échelle en y (cm par unité de donnée)

Parameters

```
resolve-origine(  
  ctx: dictionary,  
  origine: string array,  
  sx: float int,  
  sy: float int  
) -> array
```

ctx dictionary

Contexte cetz courant (fourni par get-ctx).

origine `string` or `array`

Ancre “plot.”, ou (x, y) en unités de données.

sx `float` or `int`

Facteur d'échelle en x (cm par unité de donnée).

sy `float` or `int`

Facteur d'échelle en y (cm par unité de donnée).

rotate-point

Fait tourner le point (x, y) de angle autour du centre (cx, cy).

- cx (float, int): abscisse du centre de rotation
- cy (float, int): ordonnée du centre de rotation
- x (float, int): abscisse du point à faire tourner
- y (float, int): ordonnée du point à faire tourner
- angle (angle): angle de rotation

Parameters

```
rotate-point(  
  cx: float int,  
  cy: float int,  
  x: float int,  
  y: float int,  
  angle: angle  
) -> array
```

cx `float` or `int`

Abscisse du centre de rotation.

cy `float` or `int`

Ordonnée du centre de rotation.

x `float` or `int`

Abscisse du point à faire tourner.

y `float` or `int`

Ordonnée du point à faire tourner.

angle angle

Angle de rotation.

setAxes

Dictionnaire de style des axes, à passer à `set-style(axes: ...)` (via `newFig(axes: ...)`).

- `grid` (stroke): style de la grille principale
- `minor-grid` (stroke): style de la sous-grille
- `shared-zero` (content): étiquette affichée à l'origine commune des deux axes
- `padding` (length): dépassement au-delà des axes
- `overshoot` (length): dépassement des flèches des axes
- `tick-stroke` (stroke): couleur/épaisseur des ticks majeurs
- `tick-length` (float, int): longueur des ticks majeurs
- `tick-minor-stroke` (stroke): couleur/épaisseur des ticks mineurs
- `tick-minor-length` (float, int): longueur des ticks mineurs

Parameters

```
setAxes(  
  grid: stroke ,  
  minor-grid: stroke ,  
  shared-zero: content ,  
  padding: length ,  
  overshoot: length ,  
  tick-stroke: stroke ,  
  tick-length: float int ,  
  tick-minor-stroke: stroke ,  
  tick-minor-length: float int  
) -> dictionary
```

grid stroke

Style de la grille principale.

Default: (stroke: gray + 0.4pt)

minor-grid stroke

Style de la sous-grille.

Default: (stroke: gray.lighten(60%) + 0.2pt)

shared-zero content

Étiquette affichée à l'origine commune des deux axes.

Default: text(size: 8pt, fill: red)[\$0\$]

padding length

Dépassement au-delà des axes.

Default: 0pt

overshoot length

Dépassement des flèches des axes.

Default: 8pt

tick-stroke stroke

Couleur/épaisseur des ticks majeurs.

Default: black + 0.5pt

tick-length float or int

Longueur des ticks majeurs.

Default: 0.1

tick-minor-stroke stroke

Couleur/épaisseur des ticks mineurs.

Default: gray + 0.3pt

tick-minor-length float or int

Longueur des ticks mineurs.

Default: 0

setExtraStyles

Styles supplémentaires (flèches des axes), à fusionner avec ceux de setAxes() via set-style(axes: axes + extraStyles).

- x (dictionary): style de la pointe de flèche de l'axe des abscisses
- y (dictionary): style de la pointe de flèche de l'axe des ordonnées

Parameters

```
setExtraStyles(  
  x: dictionary ,  
  y: dictionary  
) -> dictionary
```


x dictionary

Style de la pointe de flèche de l'axe des abscisses.

Default: (mark: (end: "stealth", fill: black),)

y dictionary

Style de la pointe de flèche de l'axe des ordonnées.

Default: (mark: (end: "stealth", fill: black))

setPlot

Options par défaut de `plot.plot(...)`, sous forme de dictionnaire. Peut être appelée telle quelle (valeurs “génériques”), mais `newFig` l'appelle en lui injectant SES PROPRES `size/x-min/x-max/y-min/y-max` (et `x-format/y-format` en fonction de repere:), de sorte que tu n'as jamais à répéter ces valeurs à deux endroits. Tout paramètre supplémentaire non prévu explicitement (ex. `y-equal`:) passe tel quel via `...more` et sera transmis à `plot.plot`.

- `size` (array): taille physique du plot (largeur, hauteur), en cm
- `x-min` (float, int): borne inférieure du domaine en x
- `x-max` (float, int): borne supérieure du domaine en x
- `y-min` (float, int): borne inférieure du domaine en y
- `y-max` (float, int): borne supérieure du domaine en y
- `name` (str): nom du plot (pour “plot.” dans `plot.add-anchor`)
- `x-tick-step` (float, int): pas entre deux graduations majeures en x
- `x-minor-tick-step` (float, int): pas entre deux graduations mineures en x
- `y-tick-step` (float, int): pas entre deux graduations majeures en y
- `y-minor-tick-step` (float, int): pas entre deux graduations mineures en y
- `axis-style` (str): style des axes (“school-book”, etc.)
- `x-format` (function): mise en forme des valeurs affichées sur l'axe des x
- `y-format` (function): mise en forme des valeurs affichées sur l'axe des y
- `x-label` (content): étiquette de l'axe des x
- `y-label` (content): étiquette de l'axe des y
- `x-grid` (str, none): grille verticale (“major”, “minor”, “both” ou none)
- `y-grid` (str, none): grille horizontale (“major”, “minor”, “both” ou none)

Parameters

```
setPlot(  
    size: array ,  
    x-min: float int ,  
    x-max: float int ,  
    y-min: float int ,  
    y-max: float int ,  
    name: string ,  
    x-tick-step: float int ,  
    x-minor-tick-step: float int ,  
    y-tick-step: float int ,  
    y-minor-tick-step: float int ,  
    axis-style: string ,  
    x-format: function ,  
    y-format: function ,  
    x-label: content ,  
    y-label: content ,  
    x-grid: string none ,  
    y-grid: string none ,  
    ..more: any  
) -> dictionary
```

size array

Taille physique du plot (largeur, hauteur), en cm.

Default: (12, 6.5)

x-min float or int

Borne inférieure du domaine en x.

Default: -3

x-max float or int

Borne supérieure du domaine en x.

Default: 3

y-min float or int

Borne inférieure du domaine en y.

Default: -3.5

y-max float or int

Borne supérieure du domaine en y.

Default: 3

name `string`

Nom du plot (pour “plot.” dans plot.add-anchor).

Default: `"plot"`

x-tick-step `float` or `int`

Pas entre deux graduations majeures en x.

Default: `1`

x-minor-tick-step `float` or `int`

Pas entre deux graduations mineures en x.

Default: `0.1`

y-tick-step `float` or `int`

Pas entre deux graduations majeures en y.

Default: `1`

y-minor-tick-step `float` or `int`

Pas entre deux graduations mineures en y.

Default: `0.1`

axis-style `string`

Style des axes (“school-book”, etc.).

Default: `"school-book"`

x-format `function`

Mise en forme des valeurs affichées sur l’axe des x.

Default: `v => if v != 1 { text(size: 8pt)[#v] }`

y-format `function`

Mise en forme des valeurs affichées sur l’axe des y.

Default: `v => if v != 1 { text(size: 8pt)[#v] }`

x-label `content`

Étiquette de l'axe des x.

Default: `text(size: 0.8em)[$x$]`

y-label `content`

Étiquette de l'axe des y.

Default: `text(size: 0.8em)[$y$]`

x-grid `string` or `none`

Grille verticale ("major", "minor", "both" ou none).

Default: `"both"`

y-grid `string` or `none`

Grille horizontale ("major", "minor", "both" ou none).

Default: `"both"`

..more `any`

Paramètres supplémentaires transmis tels quels à `plot.plot`.

mesaxes `function`

Alias pour compatibilité avec l'ancien nom.